

八股-前端 II

Created by	 katarrena zhao
Created time	@November 8, 2025 7:25 PM
Tags	

性能优化

主要围过程（网络，渲染），结果（时间，体积）

1.白屏时间

浏览器在渲染页面，白屏时间长，渲染时间长

HTML中JS阻断DOM树构造，因为浏览器认为JS会修改DOM树，必须等待JS代码执行完毕，恢复DOM树构造过程。所有JS都涉及DOM属性

浏览器必须等待样式表加载完成开始加载CSSOM树

（3）（危害最大）解析HTML遇到JS，CSSOM树还未构建完成，浏览器暂停脚本执行（DOM树构建过程被暂停阻塞），直到CSS样式文件下载完成，完成CSSOM树构建，重新恢复原来的解析（浏览器安全解析策略）

2.首屏时间

Chrome中，通过 Performance API /开发者工具来查看这两个页面加载性能指标（First Paint，First Contentful Pain）

```
performance.getEntriesByType('paint').forEach(entry =>
{
  console.log(entry.name + ' occurs at ' +
entry.startTime);
});
```

3.pageload 页面整体加载完成时间

```
windows.addEventListener('load',function() {
//window.onload 页面全部资源加载完才执行
})

document.addEventListener('DOMContentLoaded', function(){
//DOMContentLoaded DOM渲染完就可执行，此时图片视频可能还没加载完
})
```

Souders 在《高性能网站建设指南》一书中总结了 12 条基本规则：

- 尽量减少 HTTP 请求。
- 使用 CDN。
- 静态资源使用 Cache。
- 启用 Gzip 压缩。
- JavaScript 脚本尽量放在页面底部。
- CSS 样式表放在顶部。
- 避免 CSS 表达式。
- 减少内联 JavaScript 和 CSS 的使用，尽可能使用外部的 JavaScript 和 CS
- 减少 DNS 查询。
- 精简 JavaScript。
- 避免重定向。
- 删除重复的脚本。

开发阶段

SSR比SPA应用性能更好

类型	体积	质量	兼容	请求	压缩	透明	场景
JPG	小	中	高	是	有损	不支持	背景图、轮播图、色彩丰富图
PNG	大	高	高	是	无损	支持	图标、透明图
SVG	小	高	高	是	无损	支持	图标、矢量图
WebP	小	中	低	是	兼备	支持	看兼容情况
Base64	看情况	中	高	否	无损	支持	图标

在发布项目到生产前使用图像压缩工具处理

图像压缩工具

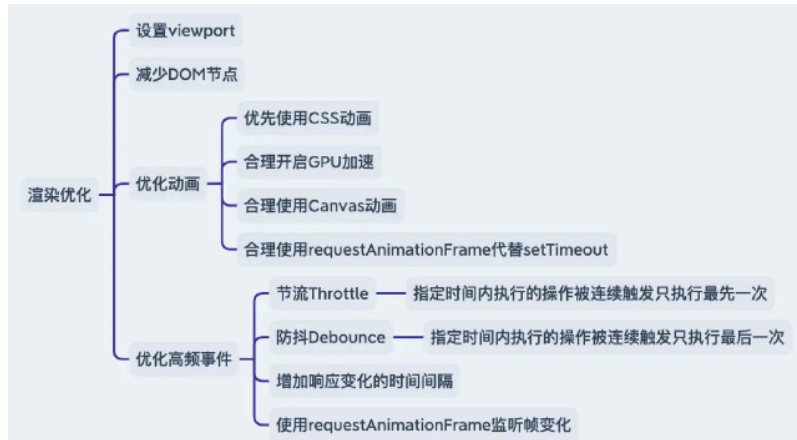
工具	开源	收费	API	免费体验
QuickPicture	✗	✓	✗	可压缩类型较多，压缩质感较好，有体积限制，有数量限制
ShrinkMe	✗	✗	✗	可压缩类型较多，压缩质感一般，无数量限制，有体积限制
Squoosh	✓	✗	✓	可压缩类型较少，压缩质感一般，无数量限制，有体积限制
TinyJpg	✗	✓	✓	可压缩类型较少，压缩质感很好，有数量限制，有体积限制
TinyPng	✗	✓	✓	可压缩类型较少，压缩质感很好，有数量限制，有体积限制
Zhitu	✗	✗	✗	可压缩类型一般，压缩质感一般，有数量限制，有体积限制

阻塞

脚本与DOM/其它脚本的依赖关系强：对<script> 设置defer

脚本与DOM/其它脚本的依赖关系不强：对<script> 设置async

DNS预解析是一种优化技术，通过提前获取域名对应的IP地址，可以加速页面加载速度。当我们在网页中使用 <link> 标签的rel 属性设置为dns-prefetch 时，浏览器会提前进行DNS 解析，以便在需要时能够更快地建立连接。



长列表优化

1.时间分片

A.使用定时器，渲染过程分批进行

```
//需要插入的容器
let ul = document.getElementById('container');
// 插入十万条数据
let total = 100000;
// 一次插入 20 条
let once = 20;
//总页数
let page = total/once
//每条记录的索引
let index = 0;
//循环加载数据
function loop(curTotal,curIndex){
  if(curTotal <= 0){
    return false;
  }
  //每页多少条
  let pageCount = Math.min(curTotal , once);
  setTimeout(() =>{
    for(let i = 0; i < pageCount; i++){
```

```

let li = document.createElement('li');
li.innerText = curIndex + i + ' : ' + ~~
(Math.random() * total)
ul.appendChild(li)
}
loop(curTotal - pageCount, curIndex + pageCount)
}, 0)
}
loop(total, index);

```

```

//需要插入的容器
let ul = document.getElementById('container');
// 插入十万条数据
let total = 100000;
// 一次插入 20 条
let once = 20;
//总页数
let page = total/once
//每条记录的索引
let index = 0;
//循环加载数据
function loop(curTotal, curIndex){
if(curTotal <= 0){
return false;
}
//每页多少条
let pageCount = Math.min(curTotal , once);
window.requestAnimationFrame(function(){
for(let i = 0; i < pageCount; i++){
let li = document.createElement('li');
li.innerText = curIndex + i + ' : ' + ~~
(Math.random() * total)
ul.appendChild(li)
}
}

```

```

loop(curTotal - pageCount, curIndex + pageCount)
})
}
loop(total, index);

```

(3) 使用 `document.createDocumentFragment` 方法或者构造函数来创建一个空的 `DocumentFragment`

```

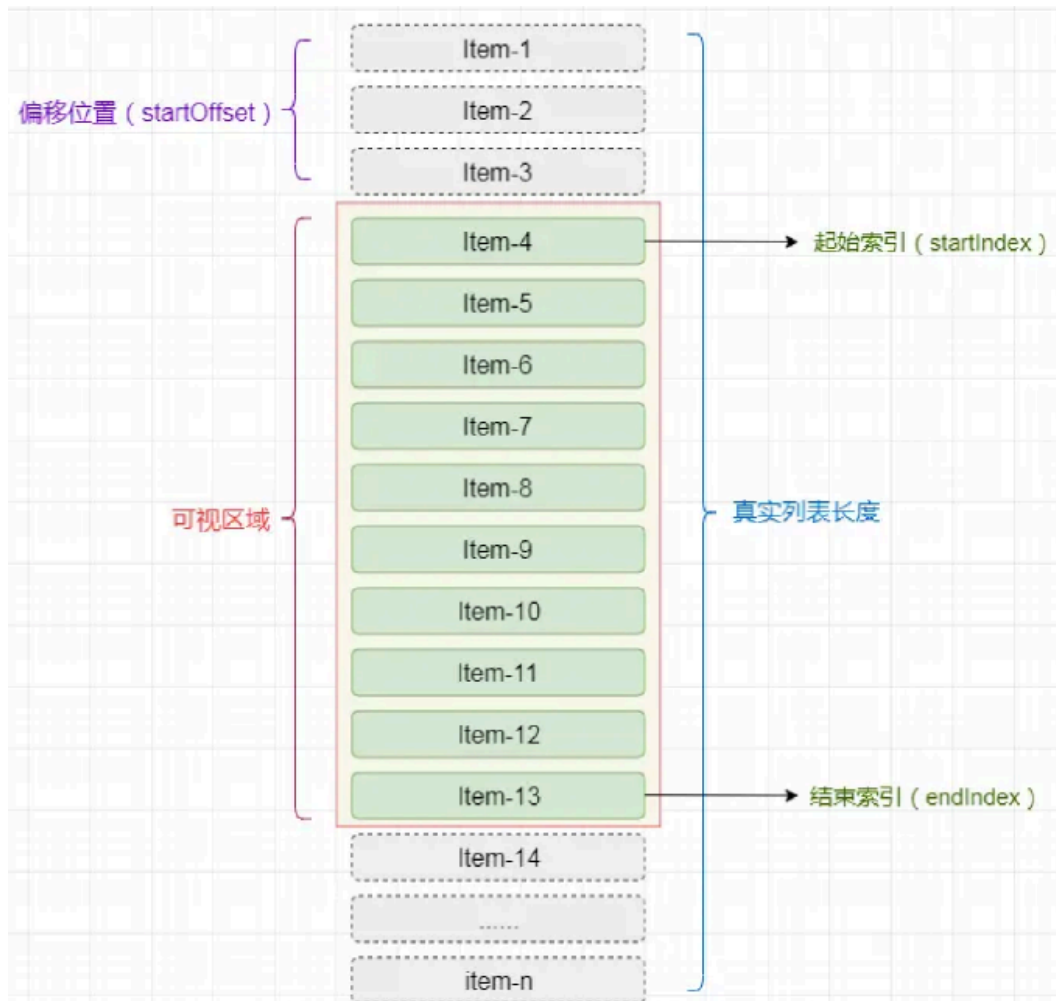
//需要插入的容器
let ul = document.getElementById('container');
// 插入十万条数据
let total = 100000;
// 一次插入 20 条
let once = 20;
//总页数
let page = total/once
//每条记录的索引
let index = 0;
//循环加载数据
function loop(curTotal, curIndex){
if(curTotal <= 0){
return false;
}
//每页多少条
let pageCount = Math.min(curTotal , once);
window.requestAnimationFrame(function(){
let fragment = document.createDocumentFragment();
for(let i = 0; i < pageCount; i++){
let li = document.createElement('li');
li.innerText = curIndex + i + ' : ' + ~~
(Math.random() * total)
fragment.appendChild(li)
}
ul.appendChild(fragment)
loop(curTotal - pageCount, curIndex + pageCount)
})
}

```

```
}  
loop(total,index);
```

2.虚拟列表

只对可见区域渲染



保证列表容器正常触发滚动

```
<div class="infinite-list-container"> //可视区域  
<div class="infinite-list-phantom"></div> //phantom 容器内占位，高度为总列表高度，用于形成滚动条
```

```

<div class="infinite-list"> //列表项渲染区域
  <!-- item-1 -->
  <!-- item-2 -->
  <!-- ..... -->
  <!-- item-n -->
</div>
</div>

```

当前滚动位置 scrollTop

假设列表每项高度固定，itemSize

推算出：

visibleCount = Math.ceil(screenHeight / itemSize)

startIndex = Math.floor(scrollTop / itemSize)

endIndex = startIndex + visibleCount

visibleData = listData.slice(startIndex,endIndex)

滚动后，渲染区域相比可视区域发生偏移，获取偏移量**startOffset**，通过样式空值将渲染区域偏移至 可视区域

偏移量startOffset = scrollTop - (scrollTop % itemSize);

骨架屏

页面开发先实现空态和加载态，后端给一个请求接口返回参数

sending = 0 报错 空态

sending =1 加载页面

打包阶段

配置include/exclude缩小Loader对文件的搜索范围，避免不必要转译

配置cache缓存Loader对文件的编译副本


```
import EslintPlugin from "eslint-webpack-plugin";
export default {
  // ...
  module: {
    rules: [{
      // ...
      test: /\.js$/,
      use: [{
        loader: "babel-loader",
        options: { cacheDirectory: true }
      }]
    }],
    plugins: [
      new EslintPlugin({ cache: true })
    ]
  };
};
```

配置resolve提高文件的搜索速度

```
export default {
  // ...
  resolve: {
    alias: {
      "#": AbsPath(""), // 根目录快捷方式
      "@": AbsPath("src"), // src目录快捷方式
      swiper: "swiper/js/swiper.min.js"
    }, // 模块导入快捷方式
    extensions: [".js", ".ts", ".jsx", ".tsx", ".json",
      ".vue"] // import路径时文件可省略后缀名
  }
};
```

threadloader帮助webpack同一时刻处理多个任务，根据CPU个数开启线程

```
import Os from "os";
export default {
  // ...
  module: {
    rules: [{
      // ...
      test: /\.js$/,
      use: [{
        loader: "thread-loader",
        options: { workers: Os.cpus().length }
      }, {
        loader: "babel-loader",
        options: { cacheDirectory: true }
      }]
    }]
  }
};
```

配置BundleAnalyzer分析打包文件结,可以找出导致体积过大的原因

BundleAnalyzer是webpack插件，分析打包文件的模块组成部分、模块体积占比、模块包含关

系、模块依赖关系、文件是否重复、压缩体积对比等可视化数

```
import { BundleAnalyzerPlugin } from "webpack-bundleanalyzer";
export default {
  // ...
  plugins: [
    // ...
    BundleAnalyzerPlugin()
  ]
};
```

分割模块代码，提取相同部分代码

```

export default {
  // ...
  optimization: {
    runtimeChunk: { name: "manifest" }, // 抽离
    WebpackRuntime函数
    splitChunks: {
      cacheGroups: {
        common: {
          minChunks: 2,
          name: "common",
          priority: 5,
          reuseExistingChunk: true, // 重用已存在代码块
          test: AbsPath("src")
        },
        vendor: {
          chunks: "initial", // 代码分割类型
          name: "vendor", // 代码块名称
          priority: 10, // 优先级
          test: /node_modules/ // 校验文件正则表达式
        }
      }, // 缓存组
      chunks: "all" // 代码分割类型：all全部模块，async异步模
    }, // 代码块分割
  },
};

```

摇树优化:删除项目中未被引用代码?

动态垫片：通过垫片服务根据UA返回当前浏览器代码垫片？

作用提升roll up：分析模块间依赖关系，把打包好的模块合并到一个函数中

在未开启作用提升前，构建后的代码会存在大量函数闭包。由于模块依赖，通过webpack 打包后会转换成IIFE，大量函数闭包包裹代码会导致打包体积增大(模块越多越明显)。运行代码时创建的函数作用域变多，从而导致更大内存开销。

开启作用提升后，构建后的代码会按照引入顺序放到一个函数作用域里，通过适当重命名某些变量以防止变量名冲突，从而减少函数声明和内存花销

压缩HTML/CSS/JS代码，压缩字体/图像/音频/视频

网站性能测试工具

Parse HTML（解析HTML）：

Scripting（脚本执行）

Recalculate Style and Layout（重新计算样式和布局）：

Paint（绘制）

LightHouse：性能评估，可访问性，最佳实践审查，SEO分析

优化原理

DNS预解析：提前获取域名对应的IP地址，加速页面加载速度。当我们在网页中使用<link> 标签的rel 属性设置为dns-prefetch 时，浏览器提前 DNS 解析，以便在需要时能够更快地建立连接。

浏览器发起请求-本地 DNS 服务器查询-根域名服务器查询-顶级域名服务器查询-权威域名服务器查询-浏览器访问网站

CDN原理（Content Delivery Network）：内容分发网络

本地DNS服务器去请求CDN的全局负载均衡器解析域

缓存策略和刷新机制

缓存过期机制

预热缓存

自动刷新

SPA vs SSR（server side render）

SPA 一定是 CRS

SPA和SSG各自优缺点？

同构：让一份代码，既可以在服务端中执行，也可以在客户端中执行，服务端渲染完成页面结构，客户端进行事件绑定和交互逻辑激活

在服务器端渲染（SSR）React 应用时，通常会使用 ReactDOM.renderToString 将 React 组件渲染为 HTML 字符串，然后将该字符串发送给客户端。客户端接收到 HTML 字符串后，可以使用 ReactDOM.hydrateRoot 激活（hydrate），将服务端生成静态 HTML 转换为具有交互性客户端应用

服务器端

```
const express = require("express");
import React from "react";
import ReactDOM from "react-dom/server";
import { StaticRouter } from "react-router-dom/server";
import { Provider } from "react-redux";
import store from "../store/index";
import App from "../app.jsx";
const server = express();
// 静态资源的部署
server.use(express.static("build"));
server.get("/*", (req, res) => {
  // 就是服务器端渲染: /about
  const AppHtmlString = ReactDOM.renderToString(
    <Provider store={store}>
    <StaticRouter location={req.url}>
    <App />
    </StaticRouter>
    </Provider>
  );
  res.send(`
    <!DOCTYPE html>
```

```

<html lang="en">
<head>
<meta charset="UTF-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
<title>Document</title>
</head>
<body>
<h1>React18 + SSR</h1>
<div id="root">${AppHtmlString}</div>
<script src="/client/client_bundle.js"></script>
</body>
</html>
`);
});
server.listen(8080, () => {
console.log("server start on 8080");
});

```

客户端

```

import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import { Provider } from "react-redux";
import store from "../store/index";
import App from "../app.jsx";
// 在需要激活的模式下挂载应用 (hydrateRoot)
ReactDOM.hydrateRoot(
document.getElementById("root"),
<Provider store={store}>
<BrowserRouter>
<App />
</BrowserRouter>

```

```
</Provider>  
);
```

React路由lazy loading

异步加载，需要什么加载什么

SPA项目如果不做处理，项目打包后，所有页面打包成文件，用户打开首页一次性加载所有资源，造成首页加载白屏时间过长

实现：

配置react路由时对组件进行 `React.lazy()` 加载，参数是一个回调，回调的返回值是一个 `import()` 方法实现异步加载。用 `Suspense` 组件包裹懒加载的组件。在 `Suspense` 组件可以用 `fallback` 参数来指定加载动画，这也是白屏性能优化的一个方法

原理：

1.代码分割。

当 Webpack 解析到 `import()` 语法时，会自动进行代码分割。代码分割指的是将打包后的 JavaScript 代码划分成更小、更独立的代码块，实现按需加载。

核心：ECMA规范规定了 `import()` 的异步加载功能，本质上来说是调用 `import` 方法返回的是一个 `promise` 对象，创建方法内部通过动态创建 `script` 元素并监听其加载状态，根据是否加载成功来改变 `promise` 的状态是 `fulfilled` 还是 `rejected`

2.React.lazy()

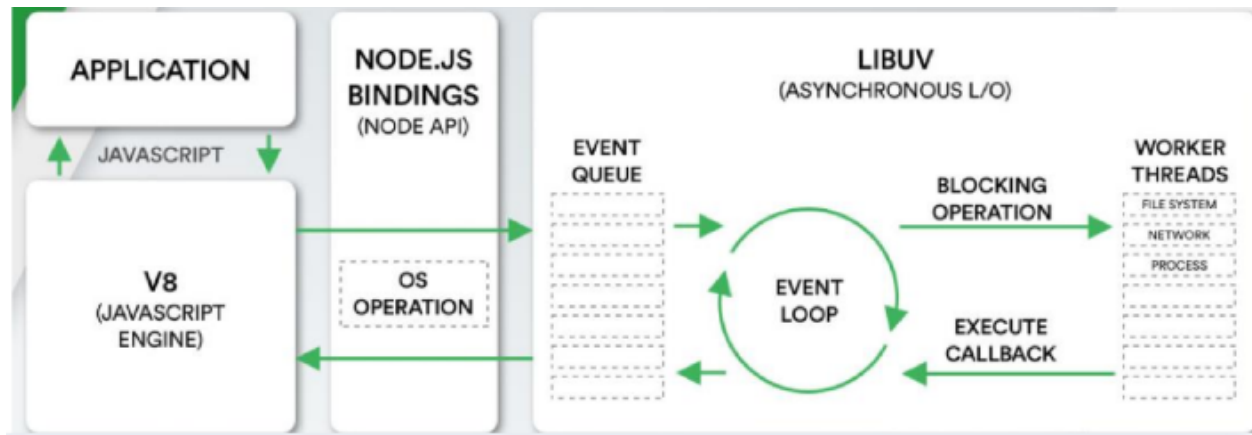
处理这个 `promise` 对象。如果 `promise` 对象已经 `Resolved` 了（说明模块加载完毕）则直接返回 `moduleObject.default`（动态加载的模块的默认导出），否则抛出异常，`React Suspense` 组件会捕获到这个异常

3.Suspense组件

捕获到异常之后会进行判断，如果处于 `pending` 状态，则会将该模块都渲染成 `fallback` 的值，一旦检测到 `resolve` 则 `Suspense` 组件包裹的子组件会重新渲染。

前端工程化 webpack优化

nodejs：基于V8引擎的运行环境



应用：

npm, yarn, pnpm成为前端开发用最多的工具

桌面应用程序开发用Electron

❑ **LTS版本**：（Long-term support, 长期支持）相对稳定一些，推荐线上环境使用该版本；

❑ **Current版本**：最新的Node版本，包含很多新特性；

[Download](#)

全局对象：

浏览器的全局对象是window，并且用var定义的变量会成为window的属性

node的全局对象是global，但是用var定义的变量不会成为全局对象的属性

Node.js 中的 process 是一个全局对象，提供当前Node.js进程信息，对其空值。

process对象有很多方法和属性处理进程，执行系统级操作

获取命令行参数

设置环境变量

退出进程

监听进程事件

设置进程标题
发送信号给进程

模块化

CommonJS

- 我们需要知道CommonJS是一个规范，最初提出是在浏览器以外的地方使用，并且当时被命名为ServerJS，后来为了体现它的广泛性，修改为CommonJS，平时我们也会简称为CJS。
 - Node是CommonJS在服务器端一个具有代表性的实现；
 - Browserify是CommonJS在浏览器中的一种实现；
 - webpack打包工具具备对CommonJS的支持和转换；
- 所以，Node中对CommonJS进行了支持和实现，让我们在开发node的过程中可以方便的进行模块化开发：
 - 在Node中每一个js文件都是一个单独的模块；
 - 这个模块中包括CommonJS规范的核心变量：exports、module.exports、require；
 - 我们可以使用这些变量来方便的进行模块化开发；
- 前面我们提到过模块化的核心是导出和导入，Node中对其进行了实现：
 - exports和module.exports可以负责对模块中的内容进行导出；
 - require函数可以帮助我们导入其他模块（自定义模块、系统模块、第三方库模块）中的内容；

AMD: Asynchronous Module Definition 异步加载模块

require.js, curl.js

CMD:Common Module Definition

SeaJS

- ES Module的解析过程可以划分为三个阶段：
 - 阶段一：构建（Construction），根据地址查找js文件，并且下载，将其解析成模块记录（Module Record）；
 - 阶段二：实例化（Instantiation），对模块记录进行实例化，并且分配内存空间，解析模块的导入和导出语句，把模块指向对应的内存地址。
 - 阶段三：运行（Evaluation），运行代码，计算值，并且将值填充到内存地址中；

ES6 vs CommonJS ?

导入和导出语法：ES6模块使用import和export语法进行导入和导出，而CommonJS模块使用require()和module.exports语法。

静态加载和动态加载：ES6模块使用静态加载，其中导入和导出语句在编译时解析，使得模块的依赖关系可以在编译时确定。而CommonJS模块使用动态加载，导入语句在运行时执行。这是因为CJS导入导出的逻辑是对象的引用赋值

异步加载：ES6模块支持异步加载模块（使用import()函数），这使得加载模块更加灵活。CommonJS模块在加载时是同步的，无法异步加载模块。

缓存值：ES6采用实时绑定的方式，导入导出的值都指向同一个内存地址，所以导入值会跟随导出值变化，并且不会缓存值，CommonJS在首次加载模块时后缓存，后续再次加载会执行缓存的结果

```
git log //查看提交历史 按照提交的时间顺序列出每个提交信息，包括提交者、提交日期、提交信息等
```

```
git reflog //查看引用日志 记录了 HEAD 和分支引用的变动历史
//引用日志会记录 HEAD 和分支的移动、重置、合并等操作，以及提交历史的哈希值。（HEAD指向当前分支的最后一次提交）
//帮助开发者回退到之前的某个状态，即使已经丢失了对应的提交记录
```

```
//Git 创建一个新的分支并切换到这个分支上
git checkout -b <branch_name>
//删除远程仓库上的分支
git push origin --delete branch_name
//切换分支
git checkout main
```

远程仓库

当使用 git clone 命令克隆一个远程仓库时，Git 会自动将远程仓库添加为默认远程仓库，并且给这个远程仓库起一个默认的名称 origin



管理远程服务器

- 查看远程地址：比如我们之前从GitHub上clone下来的代码，它就是有自己的远程仓库的：

```
git remote
git remote -v
-v是-verbose的缩写(冗长的)
```

```
$ git remote
origin
```

```
$ git remote -v
origin https://github.com/coderwhy/hy-react-web-music.git (fetch)
origin https://github.com/coderwhy/hy-react-web-music.git (push)
```

- 添加远程地址：我们也可以继续添加远程服务器（让本地的仓库和远程服务器仓库建立连接）：

```
git remote add <shortname> <url>
git remote add gitlab http://152.136.185.210:7888/coderwhy/gitremotedemo.git
```

```
$ git remote -v
gitlab http://152.136.185.210:7888/coderwhy/gittestssh.git (fetch)
gitlab http://152.136.185.210:7888/coderwhy/gittestssh.git (push)
origin https://github.com/coderwhy/hy-react-web-music.git (fetch)
origin https://github.com/coderwhy/hy-react-web-music.git (push)
```

- 重命名远程地址： `git remote rename gitlab glab`

- 移除远程地址： `git remote remove gitlab`

箭头函数 vs 普通

箭头没有自己的this，在作用域上一层继承this，没有this绑定，必须通过查找作用域链决定值

箭头不是构造函数，没有自己的arguments

call(this绑定对象, 传入参数执行参数), apply(), bind() 不能改变箭头中this指向

箭头没有prototype

箭头不能用作generator函数，不能使用yield关键字

显式绑定 隐式丢失

//1.变量赋值,我们直接把一个对象的方法赋值给一个变量,这个变量成为了一个函数,然后调用这个变量,发现this指向全局或者undefined

```
var a = 1;
```

```

function foo() {
  var a = 3;
  console.log(this.a)
}
var obj = {
  a:2,
  foo:foo
};
var bar = obj.foo;
bar() // 1
//2.参数传递,直接把一个对象的方法传递给一个函数作为函数的参数
var a = 1;
function foo() {
  var a = 3;
  console.log(this.a)
}
function doFun(fn) {
  fn()
}
var obj = {
  a:2,
  foo:foo
};
doFun(obj.foo;) // 1

```

执行上下文

this绑定 → lexicalEnvironment → VariableEnvironment

词法环境：环境记录器记录变量+Outer对象指向父级作用域

异步编程

promise

```

class myPromise {
  state = "pending";

```

```

value = undefined;
message = undefined;
constructor(fn){
  const resolveHandler= (value) =>{
    this.state = "resolved";
    this.value =value;
  }
  const rejectHanler = (message) => {
    this.state = "rejected";
    this.message = message;
  }
  try {
    fn(resolveHandler,rejectHanler);
  }
  catch (error){
    rejectHanler(error)
  }
}
then(fn1,fn2) {
  if (this.state === "resolved"){
    return new myPromise ((resolve,reject) =>{
      const v = fn1(this.value);
      resolve(v);
    })
  }
  if (this.state === "rejected"){
    return new myPromise ((resolve,reject) =>{
      const m = fn2(this.message);
      resolve(m);
    })
  }
}
//一些静态方法：resolve(),all(),race()
//静态方法
myPromise.resolve = (value) =>{

```

```

return new myPromise((resolve, reject) => {
  resolve(value);
})
}

myPromise.reject = (message) => {
  return new myPromise((resolve, reject) => {
    reject(message);
  })
}

MyPromise.all = function (promiseList = []) {
  const p1 = new MyPromise((resolve, reject) => {
    const result = [] // 存储 promiseList 所有的结果
    const length = promiseList.length
    let resolvedCount = 0
    promiseList.forEach(p => {
      p.then(data => {
        result.push(data)
        // resolvedCount 必须在 then 里面做 ++
        // 不能用 index
        resolvedCount++
        if (resolvedCount === length) {
          // 已经遍历到了最后一个 promise
          resolve(result)
        }
      }).catch(err => {
        reject(err)
      })
    })
  })
  return p1
}

MyPromise.race = function (promiseList = []) {
  let resolved = false // 标记
  const p1 = new Promise((resolve, reject) => {
    promiseList.forEach(p => {
      p.then(data => {

```

```

if (!resolved) {
  resolve(data)
  resolved = true
}
}).catch((err) => {
  reject(err)
})
})
})
return p1
}

```

其他API

```

const p1 = ajax("https://xiongmaoyouxuan.com/api/tabs");
const p2 = new Promise((resolve, reject) => {
  setTimeout(() => {
    reject("超时!");
  }, 2000)
})
Promise.allSettled([p1, p2]).then(res => { //确定一组异步操作是否都结束了（不管成功或者失败）
  console.log(res) // 是一个返回的数组，包括状态和返回结果
}).catch(err => {
  console.error(err)
})

const p1 = new Promise((resolve, reject) => {
  setTimeout(_ => {
    reject("11111");
  }, 1000)
})
const p2 = new Promise((resolve, reject) => {
  setTimeout(_ => {
    reject("22222");
  }, 1000)
})

```

```

    }, 2000)
  })
  const p3 = new Promise((resolve, reject) => {
    setTimeout(_ => {
      resolve("33333");
    }, 3000)
  })
  promise.any([p1, p2, p3]).then(res => {
    console.log(res);
  }).catch(err => {
    console.error(err);
  })
  // 3秒后返回'33333'

  //不管任何状态变化都会触发finally()

```

Generator

每次调用next()执行到下一个yield语句；调用generator不会立即执行函数内代码，返回一个iterator。执行generator内，通过调用iterator对象next()方法（返回有value和done属性的对象）

状态机：每个状态对应一个generator，通过不同的 yield表达式切换状态实现状态流转和逻辑处理

```

// 定义状态机的不同状态
function* stateOne() {
  console.log('In state one, performing some actions...');
  yield 'stateTwo'; // 切换到 stateTwo
}
function* stateTwo() {
  console.log('In state two, performing some other actions...');
  yield 'stateThree'; // 切换到 stateThree
}

```



```

function* stateThree() {
  console.log('In state three, performing final
  actions...');
  yield 'end'; // 结束状态机
}
// 状态机执行函数
function stateMachine() {
  let currentState = stateOne(); // 初始状态为 stateOne
  while (true) {
    const { value, done } = currentState.next();
    if (done) {
      console.log('State machine ended.');
```

```

      break;
    }
    if (value === 'stateTwo') {
      currentState = stateTwo();
    } else if (value === 'stateThree') {
      currentState = stateThree();
    } else {
      console.log('Unknown state:', value);
      break;
    }
  }
}
// 执行状态机
stateMachine();

```

数据流控制

```

// 定义一个 Generator 函数来处理数据流
function* processData() {
  console.log('Processing data...');
  // 模拟一些数据处理过程
  for (let i = 0; i < 5; i++) {
    // 模拟异步操作，例如从网络请求数据
    yield fetchData(i);
  }
}

```

```

}
console.log('Data processing complete.');
```

// 模拟异步获取数据的函数

```

function fetchData(index) {
return new Promise(resolve => {
setTimeout(() => {
resolve(`Data ${index}`);
}, Math.random() * 1000); // 模拟不同的数据获取时间
});
}
// 执行数据处理函数
async function processDataFlow() {
const dataGenerator = processData();
for await (const data of dataGenerator) {
console.log('Received data:', data);
}
}
// 执行数据流控制
processDataFlow();
```

async函数

返回promise对象

await等待的是返回值，await 等 Promise 对象和任意表达式的结果

eventloop

GUI渲染线程	负责渲染页面，解析html和CSS、构建DOM树、渲染树、和绘制页面，重绘重排也是在该线程执行
JS引擎线程	一个tab中只有一个JS引擎线程(单线程)，负责解析和执行JS。它和GUI渲染进程不能同时执行，只能一个一个来，如果JS执行过长就会导致阻塞掉帧
计时器线程	指setInterval和setTimeout，因为JS引擎是单线程，所以如果处于阻塞状态，那么计时器就会不准了，所以需要单独

	的线程来负责计时器工作
异步http请求线程	XMLHttpRequest连接后浏览器开的一个线程，比如请求有回调函数，异步线程就会将回调函数加入事件队列，等待JS引擎空闲执行
事件触发线程	主要用来控制事件循环，比如JS执行遇到计时器，AJAX异步请求等，就会将对应任务添加到事件触发线程中，在对应事件符合触发条件触发时，就把事件添加到待处理队列的队尾，等JS引擎处理。
宏任务（DOM 渲染后执行）	setTimeout, setInterval, AJAX, DOM事件
微任务（DOM渲染前执行）	Promise.then, async 函数的await

宏任务队列空，eventloop结束

垃圾回收机制

JS有自动垃圾回收机制（标记清除，引用计数）

标记清除：mark“进入环境”变量不能被回收；“离开环境”变量被内存释放

引用计数：声明变量将引用类型赋值给该变量，值引用次数为1.当引用次数=0，变量无价值，垃圾回收机制下次运行时，变量占有内存空间被释放出来

问题：相互引用的obj1和obj2循环计数，在离开作用域后引用次数不是0，引起循环引用（对象引用次数在创建时为1）

减少垃圾回收：

优化数组，清空数组，不要赋值空数组，会创建新的空对象，令length(arr)=0

优化object：对象尽量复用，不再使用对象设置为nul，尽快被回收

多使用局部变量 更快释放内存，因为局部变量生命周期比全局变量短，离开环境（函数执行计数），被标记为垃圾回收

```

//property相关api:修改DOM对象上自带属性，无法获取或修改自定义属性
const div1 = document.getElementById('div1')
console.log('div1', div1)
const divList = document.getElementsByTagName('div') // 集合
console.log('divList.length', divList.length)
console.log('divList[1]', divList[1])
const containerList =
document.getElementsByClassName('container') // 集合
console.log('containerList.length', containerList.length)
console.log('containerList[1]', containerList[1])
const pList = document.querySelectorAll('p')
console.log('pList', pList)
const pList = document.querySelectorAll('p')
const p1 = pList[0]
// property 形式
p1.style.width = '100px'
console.log( p1.style.width )
p1.className = 'red'
console.log( p1.className )
console.log(p1.nodeName)
console.log(p1.nodeType) // 1

// attribute相关api：修改dom对象上自带的属性，可以设置自定义属性，有可能引发DOM重新渲染

p1.setAttribute('data-name', 'imooc')
console.log( p1.getAttribute('data-name') )
p1.setAttribute('style', 'font-size: 50px;')
console.log( p1.getAttribute('style') )

```

DOM vs BOM Browser Object Model

阻止冒泡行为：非IE 浏览器 `stopPropagation()`，IE 浏览器 `window.`

`event.cancelBubble = true`

阻止默认行为：非 IE 浏览器 `preventDefault()`，IE 浏览器 `window. event.returnValue = false`

rest参数

Rest参数必须是函数参数列表中的最后一个参数，因为它会收集所有剩余输入参数

```
function printNames(first, second, ...others) {  
  console.log(`First: ${first}, Second: ${second}`);  
  console.log(`Others: ${others.join(', ')}`);  
}  
printNames('Alice', 'Bob', 'Charlie', 'Dave', 'Eve');  
// 输出:  
// First: Alice, Second: Bob  
// Others: Charlie, Dave, Eve
```

浅拷贝 vs 深拷贝

debouncing vs throttling

```
function throttle(func, delay) {  
  let lastExecutedTime = 0;  
  return function(...args) {  
    const currentTime = Date.now();  
    if (currentTime - lastExecutedTime > delay) {  
      func.apply(this, args);  
      lastExecutedTime = currentTime;  
    }  
  };  
}
```

```
function throttle(func, delay) {
  let timerId; // 定时器ID
  let lastExecTime = 0; // 上次执行时间
  return function() {
    const context = this; // 保存函数执行上下文
    const args = arguments; // 保存传入的参数
    const currentTime = new Date().getTime(); // 获取当前时间
    if (currentTime - lastExecTime >= delay) { // 如果距离上次执行超过指定延迟时间
      func.apply(context, args); // 执行传入的函数
      lastExecTime = currentTime; // 更新上次执行时间
    } else {
      clearTimeout(timerId); // 清除之前的定时器
      timerId = setTimeout(function() {
        func.apply(context, args); // 延迟执行传入的函数
        lastExecTime = currentTime; // 更新上次执行时间
      }, delay - (currentTime - lastExecTime)); // 计算延迟时间
    }
  };
}
```

HTML

SEO (Search Engine Optimization)

flex-direction

```
.box {
  flex-direction: row | row-reverse | column | columnreverse;
}
```

flex-wrap

```
.box{  
  flex-wrap: nowrap | wrap  
}
```

flex-flow

```
.box {  
  flex-flow: <flex-direction> <flex-wrap>;  
}
```

display属性值：

display:none

display:inline

display:block

display:list-item

display:inline-block

display:table 本身并不产生 "block formatting contexts",可以产生匿名框，包含"display：table-cell"框产生块格式化上下文

display: flex

BFC(Block Formatting Context)

触发方法

浮动元素 (float: left right)
绝对定位，固定定位 (position: absolute fixed)
flex (display: flex)
'overflow' 特性不为 visible 的元素

匿名框：块级框和行内框

BFC：块级元素，块级元素会在垂直方向一个接一个排列

圣杯布局：



header和footer各自占领屏幕所有宽度，高度固定。

中间的container是一个三栏布局。

三栏布局两侧宽度固定不变，中间部分自动填充整个区域。

中间部分的高度是三栏中最高的区域的高度。

双飞翼布局

没有使用相对定位

响应式设计 vs 自适应设计

响应式设计：响应式开发一套界面，通过检测视口分辨率，针对不同客户端在客户端做代码处理，来展现不同的布局和内容。

自适应设计：自适应需要开发多套界面，通过检测视口分辨率，来判断当前访问的设备是 PC 端、平板还是手机，从而请求服务端，返回不同的页面。

reflow 回流

render tree中DOM元素尺寸、结构等属性改变，浏览器重新渲染部分或全部文档

重绘repaint

CSS动画

transition过渡动画

animation过渡动画

webpack

代码分割：将应用程序拆分成多个文件可以减少初始加载时间，实现按需加载，并且浏览器可以缓存更改很少的模块

Hot Module Replacement, HMR 模块热替换 当某个模块变化时只会重新加载该模块，不会刷新整个页面，提高开发效率

半双工 vs 全双工

多进程打包 (loader)

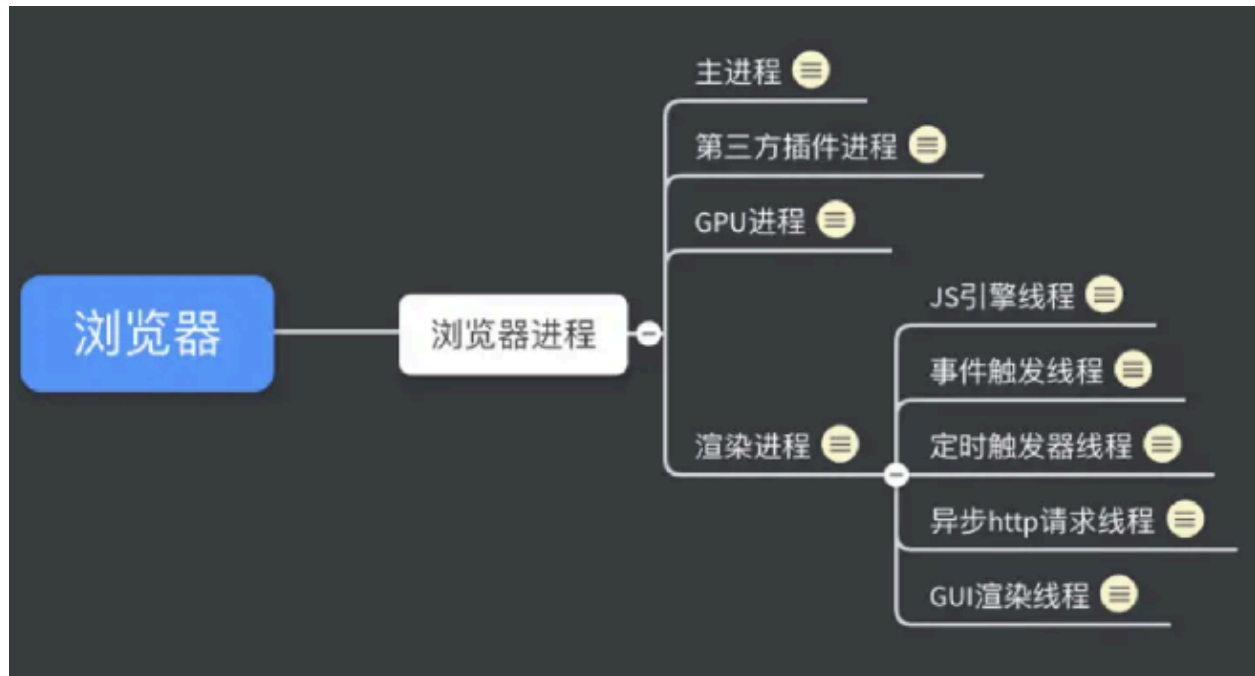
codeSplit 代码拆分为多个小块

PWA ?

- 整个目录结构都非常好理解，只是有一个PWA相关的概念：
 - PWA全称Progressive Web App，即渐进式WEB应用；
 - 一个 PWA 应用首先是一个网页，可以通过 Web 技术编写出一个网页应用；
 - 随后添加上 App Manifest 和 Service Worker 来实现 PWA 的安装和离线等功能；
 - 这种Web存在的形式，我们也称之为是 Web App；
- PWA解决了哪些问题呢？
 - 可以添加至主屏幕，点击主屏幕图标可以实现启动动画以及隐藏地址栏；
 - 实现离线缓存功能，即使用户手机没有网络，依然可以使用一些离线功能；
 - 实现了消息推送；
 - 等等一系列类似于Native App相关的功能；
- 更多PWA相关的知识，可以自行去学习更多；
 - https://developer.mozilla.org/zh-CN/docs/Web/Progressive_web_apps

浏览器

进程 vs 线程



GUI 渲染线程

JS 引擎线程

事件触发线程

定时触发器线程

setInterval setTimeout

浏览器定时计数器并不是帮JS引擎计数（JS引擎单线程，如果阻塞会影响计时准确）

异步HTTP请求线程

XMLHttpRequest在连接后是通过浏览器新开一个线程请求

检测到状态变更时，如果设置有回调函数，异步线程产生状态变更，回调放入事件队列中，由JS引擎执行

process-per-site-instance

即使是如今的多进程架构，我偶尔还会碰到一些由于单个页面卡死最终崩溃导致所有页面崩溃的情况，请问这是什么原因呢？

通常情况是一个页面使用一个进程，但是，有一种情况，叫"同一站点(same-site)"，比如下面这三个：<https://time.geekbang.org> <https://www.geekbang.org> <https://www.geekbang.org:8080> 都是属于同一站点，因为它们的协议都是https，而根域名也都是geekbang.org。Chrome的默认策略

是，每个标签对应一个渲染进程。但是如果从一个页面打开了新页面，而新页面和当前页面属于同一站点时，那么新页面会复用父页面的渲染进程。官方把这个默认策略叫process-per-site-instance。直白的讲，就是如果几个页面符合同一站点，那么他们将被分配到一个渲染进程里面去。所以，这种情况下，一个页面崩溃了，会导致同一站点的页面同时崩溃，因为他们使用了同一个渲染进程。为什么要让他们跑在一个进程里面呢？因为在一个渲染进程里面，他们就会共享JS的执行环境，也就是说A页面可以直接在B页面中执行脚本。

浏览器安全

同源策略

一个url由三部分组成:协议，域名，端口，只有当协议，域名，端口都一致，才能称为同源，同源浏览器才接受响应

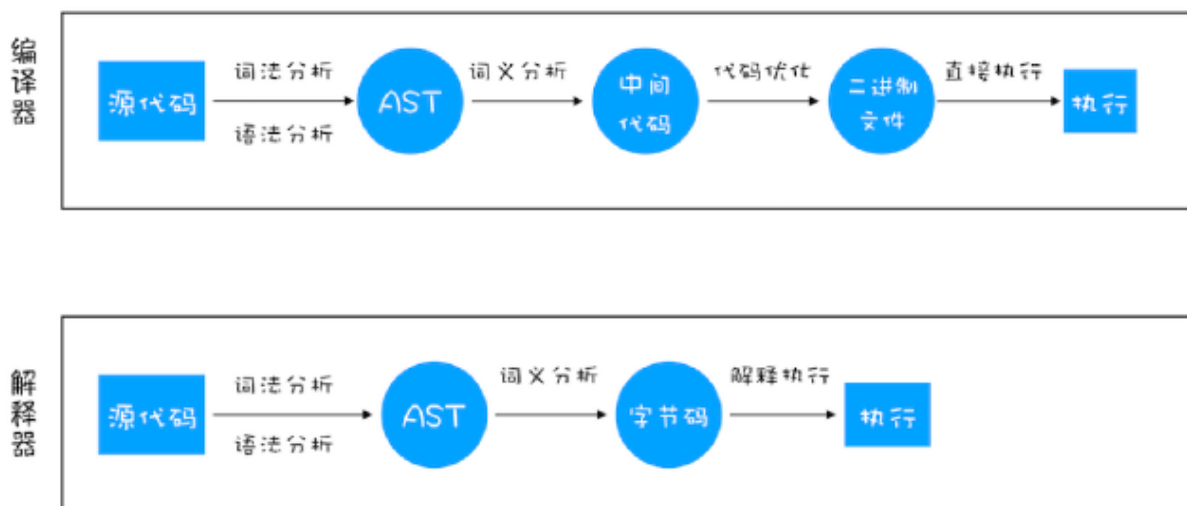
代理解决跨域

跨域问题产生前提：浏览器环境，代理服务器不受浏览器的限制，所以可以和远程服务器正常通信。

代理服务器和html网页在相同的域名、端口下，所以不存在跨域，代理服务器可以把数据传给页面

代理服务器会在请求头中添加一些额外的信息，其中包括目标服务器的主机名（Host）和客户端原始的请求信息。目标服务器收到请求后，会根据请求头中的主机名以及请求的具体路径等信息来确定客户端需要的是哪个资源，并进行相应的处理和响应

JS引擎中V8是佼佼者，chrome和node底层用的都是V8，CPU不认识js代码，不同CPU只认识自己对应的指令集。JS引擎是负责将js代码编译成CPU认识的指令集，出了编译外负责执行和内存管理。引擎读取源码，一边编译一边执行。



执行过程

抽象语法树AST，并生成执行上下文

有了AST和执行上下文后，那接下来的第二步，解释器 Ignition 就登场了，它会根据AST生成字节码，并解释执行字节码。

字节码就是介于AST和机器码之间的一种代码。但是与特定类型的机器码无关，字节码需要通过解释器将其转换为机器码后才能执行

JIT：在 Ignition 执行字节码中，如果发现有热点代码（HotSpot），比如一段代码被重复执行多次，这种就称为热点代码，那么后台的编译器 TurboFan 就会把该段热点的字节码编译为高效的机器码，然后当再次执行这段被优化的代码时，只需要执行编译后的机器码就可以了

不同源

websocket

PostMessage API

window.name

fragment identifier

axios二次封装和axios原理

```
const instance = axios.create({
  baseURL: 'https://some-domain.com/api/', //设置请求的基础URL，会被添加到相对URL之前
  timeout: 1000, //设置请求超时时间，单位是毫秒
  headers: {'X-Custom-Header': 'foobar'} //设置请求的自定义headers
});
```

根据创建实例添加请求拦截器 以及响应拦截器

拦截器use方法参数两个都是回调函数

请求拦截器第一个回调函数：在发送请求之前做点什么，第二个参数：请求错误后做什么

响应拦截器第一个回调函数：响应成功后对响应数据做什么， 第二个回调：响应错误之后做什么

```
// 请求拦截器
instance.interceptors.request.use(
  config => {
    if (config.showLoading) showLoading()
    const token = storage.get('token')
    if (token) {
      config.headers.Authorization = 'Bearer ' + token
    }
    if (env.mock) {
      config.baseURL = env.mockApi
    } else {
      config.baseURL = env.baseApi
    }
    return {

```

```

...config
}
},
(error: AxiosError) => {
return Promise.reject(error)
}
)
// 响应拦截器
instance.interceptors.response.use(
response => {
const data: Result = response.data
hideLoading()
if (response.config.responseType === 'blob') return
response
if (data.code === 500001) {
message.error(data.msg)
storage.remove('token')
location.href = '/login?callback=' +
encodeURIComponent(location.href)
} else if (data.code !== 0) {
if (response.config.showError === false) {
return Promise.resolve(data)
} else {
message.error(data.msg)
return Promise.reject(data)
}
}
return data.data
},
error => {
hideLoading()
message.error(error.message)
return Promise.reject(error.message)
}
)

```

//默认导出实例的get方法,post方法（写成对象的形式）,通常用闭包的形式对get方法和post方法做增强

```
export default {  
  get(url, params, options) {  
    return instance.get(url, { params, ...options })  
  },  
  post(url, params, options): Promise<T> {  
    return instance.post(url, params, options)  
  }  
}
```

如何封装ajax请求？

请求结果封装成promise对象



//用 Promise 封装 Ajax 请求函数

```
function ajax(url, method = 'GET', data = null) {  
  return new Promise((resolve, reject) => {  
    const xhr = new XMLHttpRequest();  
    xhr.open(method, url, true);
```

```

xhr.setRequestHeader('Content-Type',
'application/json');
xhr.onload = function() {
if (xhr.status >= 200 && xhr.status < 300) {
resolve(xhr.responseText);
} else {
reject(new Error(`请求失败：${xhr.status} -
${xhr.statusText}`));
}
};
xhr.onerror = function() {
reject(new Error('网络错误'));
};
xhr.send(data ? JSON.stringify(data) : null);
});
}
// 示例用法
ajax('https://api.example.com/data')
.then(response => {
console.log('成功获取数据：', response);
})
.catch(error => {
console.error('请求失败：', error);
});

```

Nginx (engine X) 高性能HTTP和反向代理web服务器

高并发 (high concurrency)，高性能

reverse proxy：以代理服务器接受internet连接请求，请求转发给内部网络上的服务器，将从服务器上得到的结果返回给internet上请求连接的客户端，代理服务器对外就表现为一个反向代理服务器。

window.parent vs window.top

dedicated worker vs shared worker

分布式系统 对于用户每个请求被不同服务器处理，引入挑战，处理用户对话时：

1.状态管理：在传统的单体应用中，可以将用户会话信息直接存储在服务器的内存中。但在分布式系统中，由于用户请求可能会被不同的服务器处理，因此无法简单地依赖服务器内存来管理会话状态。这就需要一种能够在不同服务器之间共享会话信息的机制。

2. 负载均衡：分布式系统通常会使用负载均衡器来分发用户请求到不同的服务器节点，以提高系统的性能和可用性。但这样做可能会导致用户的不同请求被发送到不同的服务器上，从而造成用户会话信息的不一致性。

如何选取保持用户的登录状态？

cookie

session也可以存储，把sessionID放入cookie里，性能差，不适宜分布式系统

token最常用，因为JWT有很多优缺点：

JWT使用签名（HMAC）或公钥/私钥（RSA 或 ECDSA）进行加密，确保令牌的完整性和可信度。通过对令牌进行签名或加密，可以防止令牌被篡改或伪造

JWT 适用于分布式系统和无状态服务架构，每个服务都可以独立验证JWT 的有效性，而无需依赖中心化的会话存储，性能很好

默认情况下，JWT 中的信息是以 Base64 编码的形式进行传输，虽然可以使用加密算法对整个令牌进行签名，但令牌中的信息本身并不加密。这意味着任何人都可以解码 JWT 并读取其中的信息，因此不适合传输敏感信息。

增加 CSRF（Cross-Site Request Forgery，跨站请求伪造）攻击的风险。这是因为 JWT 通常存储在客户端的cookie或会话存储中，而不像 Cookie那样可以设置 HttpOnly 属性来防止 JavaScript 访问。

一旦签发了 JWT，在到期之前就会始终有效，可能会导致令牌被滥用，恶意用户可能利用令牌来执行一些恶意操作

SSO Single Sign On

用户只需登录一次，就可以访问多个系统，而无需为每个系统单独提供凭据。

跨域解决

同源：协议、域名、端口相同

为什么要同源？

防止跨站点请求伪造（CSRF）攻击

保护用户隐私

防止跨站脚本（XSS）攻击

如何获取不同源cookie？

允许不同源的cookie之间可以互相访问的行为一般不会实现，因为安全风险太高。但是我们希望两个网页一级域名相同，只是二级域名不同，实现两个cookie共享则是一种比较合理的请求。

CORS 跨源资源共享

一种机制，允许Web服务器声明哪些源站可以通过浏览器访问哪些资源